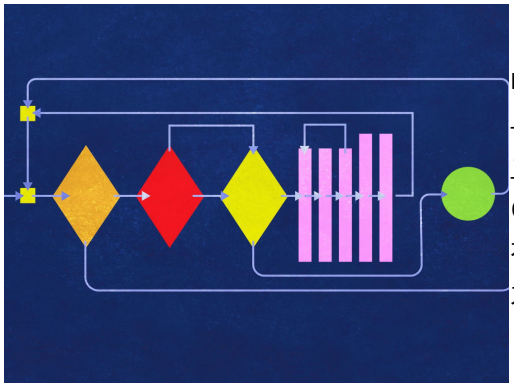
Vision Day3 보고서
머신 러닝 + PyTorch

휴머노이드 수습단원 이주원

목차

1. 머신러닝이란
2. 머신 러닝의 유형(지도/비지도/강화 학습)
3. 딥러닝
    * CNN, RNN
4. 퍼셉트론
5. 활성화 함수
6. 비용함수
7. 경사하강법
8. 역전파
9. 심층신경망

1. 머신러닝이란



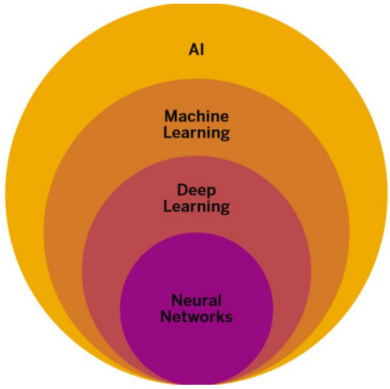
머신러닝은 학습 데이터의 패턴을 학습한 후 새로운 데이터에 대해 정확한 추론을 할 수 있는 알고리즘에 초점을 맞춘 인공지능의 하위 집합이다. 이 패턴 인식 기능을 통해 머신 러닝 모델은 명시적이고 하드 코딩된 지침 없이 결정이나 예측이 가능하다.

머신러닝의 핵심적인 전제는 모델 학습이라는 프로세스를 통해 모델이 사용될 실제 문제와 충분히 유사한 작업 데이터 세트에서 모델의 성능을 최적화하면 모델이 궁극적인 사용 사례에서 보게 되는 새 데이터를 정확하게 예측할 수 있다는 것이다.

머신러닝의 기본 목표는 학습데이터의 성능을 실제 시나리오에서 유용한 결과로 변환하는 일반화이며 학습 자체는 단순히 목적을 위한 수단이다.

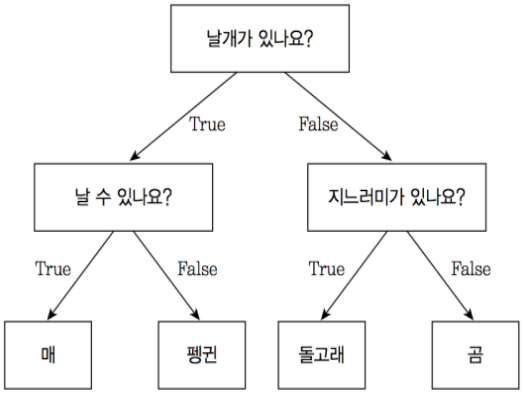
머신러닝의 용어의 어원은 Arthur L. Samuel이 1959년에 IBM저널에 기고한 체커게임을 활용한 몇 가지 러닝머신 연구로 여겨진다.

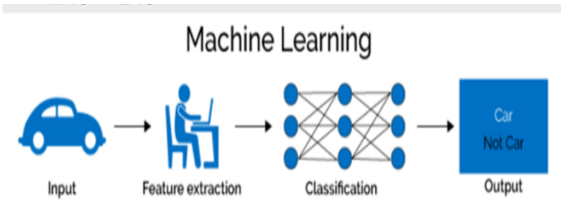
인공지능과 머신 러닝은 같은 의미로 사용될 수 있지만 동의어는 아니다. 모든 머신 러닝은 AI이지만, 모든 AI는 머신러닝이 아니다.



흔히 생각하는 AI는 터미네이터 같은 영화에서 나온 로봇으로 묘사되는데, 인공지능은 인간의 적극적인 개입 없이 정보를 사용하여 의사 결정이나 예측을 할 수 있는 모든 프로그램을 포괄하는 용어이다.

가장 기본적인 AI시스템은 데이터 과학자가 명시적으로 프로그래밍한 규칙과 로직을 포함한다. 이는 일련의 if문 같은 명령문이다. 예를 들어보면 만약 방의 온도가 10도 보다 낮다면 난방을 켜라 같은 단순한 온도 조절기 또한 규칙 기반 AI시스템이다. 온도 조절기는 추가적인 인간의 개입 없이 자율적인 의사 결정이 가능하며 더 복잡한 수준에서 전문가가 프로그래밍한 크고 복잡한 규칙 기반 의사결정트리는 삶의 질을 높이는 의료분야와 같은데서 쓰일 수 있다.





이러한 시스템과 달리 머신 러닝 모델이 작동하는 논리는 명시적으로 프로그래밍 되지 않고 경험을 통해 학습하게 된다. 예를 들어 이메일의 스팸을 필터링하는 프로그램을 생각해보자. 규칙 기반 AI는 데이터 사이언티스트가 스팸에 대한 정확하고 보편적인 기준을 스스로 찾아야 하는 방면 학습에서 모델은 샘플 이메일을 보고 스팸이 무엇인지 예측할 수 있다. AI시스템이 수행해야 하는 작업이 점점 더 복잡해진다면 규칙 기반 모델은 점점 더 취약해질 것이다.

좀 더 실제 같은 예를 들어보며 이해해보자. 주택 면적, 연식, 침실 수라는 세 가지 변수의 조합으로 주택 판매 가격을 예측하는 알고리즘이 있다. 각 주택은 3차원 벡터로 표현된다. [square footage, bedrooms, age] 연식이 30년이고 침실이 4개이며 50평인 주택은 [50, 4, 30]으로 표현될 수 있다. 이 변수를 가지는 알고리즘은

가격 = (A * 평방피트) + (B * 침실개수) - (C * 연식) + 기본가격

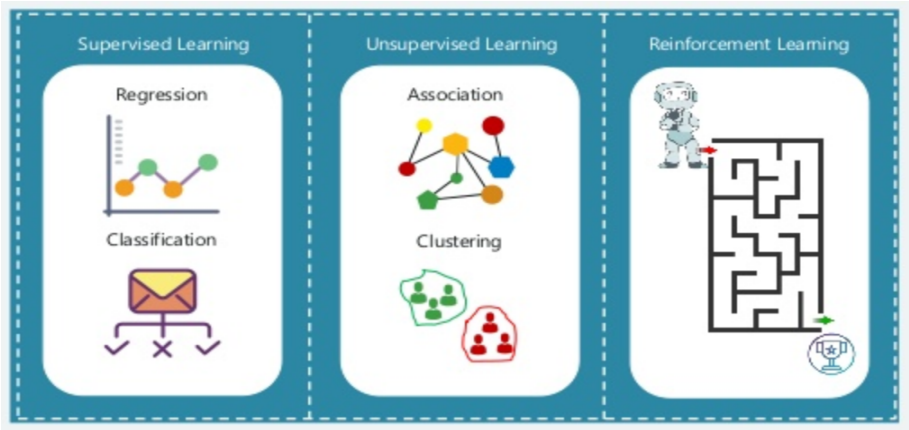
이런식으로 표현될 수 있다. 여기서 A,B,C는 모델의 매개 변수이고 이를 조정하면 모델이 각 변수에 얼마나 많은 가중치를 가하는지 조정된다. 머신 러닝의 목표는 이러한 모델 매개변수에 대한 최적의 값, 즉 전체 함수가 가장 정확한 결과를 출력하도록 하는 매개변수를 찾는 것이다. 실제 머신러닝에서는 좀 더 복잡하고 많은 매개변수가 있지만 알고리즘의 조정 가능한 매개변수를 최적하여 더 높은 정확도를 얻는 원칙은 동일하게 진행된다.

2. 머신러닝의 유형

모든 머신 러닝 방법은 학습 목표의 특성과 수반되는 학습 데이터 유형에 따라 지도학습, 비지도 학습, 강화 학습중 하나로 분류할 수 있다.

-강화학습은 주어진 질문에 대한 옳은 답변을 하도록 학습된다. 분류나 회기 같은 외부의 실제 측정된 정보와 비교하여 어느 정도의 정확도가 필요한 작업에 적용된다.
-비지도학습은 데이터의 본질적인 패턴, 종속성, 상관관계를 식별하도록 모델을 학습시킨다. 지도 학습과 달리 비지도 학습은 결과를 비교해야하는 외부 정보가 포함되어있지 않는다.

-강화학습은 모델이 주변 환경을 평가하고 가장 큰 보상을 얻을 수 있는 조치를 취하도록 학습시킨다. 이 시나리오에서는 실제 측정된 값을 가지지는 않지만 좋은 행동 혹은 나쁜 행동의 존재를 가질 수 있을 뿐만 아니라 중립적인 값을 가질수도 있다.

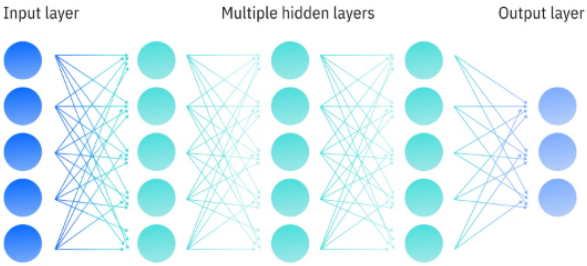


이러한 방식들을 섞어서 쓰는 하이브리드 방식이 사용되는 경우는 많다. 예를 들어 비지도 학습은 지도 학습 또는 강화 학습에 사용하기 위해 데이터를 전처리하는데 자주 사용된다. LLM은 일반적으로 초기 학습과 지도 학습의 변형을 통한 미세조정을 거친 다음 인간의 피드벡을 통한 강화학습과 같은 기술을 통해 추가로 조정한다.

3. 딥러닝

딥러닝 챕터에서는 자세한 내용이 아닌 전체적인 흐름과 딥러닝 그 자체에 대해 기고한다.

딥 러닝은 기존 머신 러닝의 명시적으로 설계된 알고리즘이 아닌, 여러 층으로 구성된 신경망을 사용한다. 인간의 뇌에서 영감을 받은 신경망은 서로 연결된 뉴런 층으로 구성 되어 있으며 각 층의 고유한 수학적 연산(활성화 함수)을 수행한다. 각 뉴런의 활성화 함수의 결과 값은 네트워크의 최종 결과값이 계산되는 최종 계층까지 다음 계층의 각 노드에 대한 입력 역할을 한다. 가장 중요한 점은 각 뉴런에서 수행되는 활성화 함수가 비선형으로 신경망이 복잡한 패턴과 종속성을 모델링할 수 있다는 것이다.

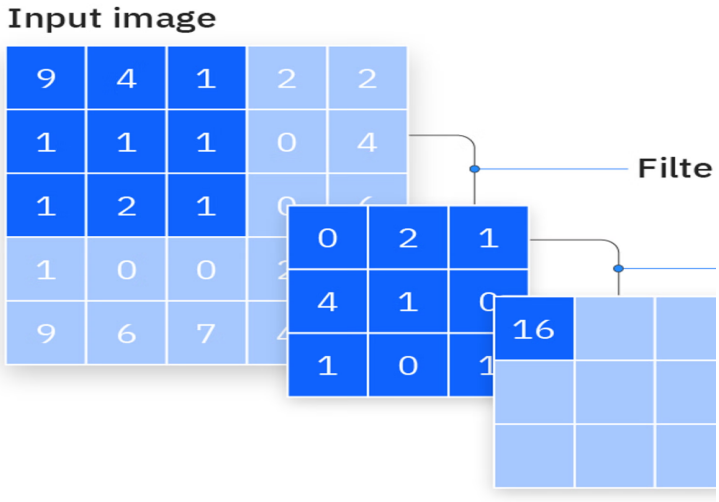


두 뉴런 사이의 각 연 결에는 다음 계층의 뉴런에 대한 한 뉴런의 기여도를 증가 또는 감소시키는 가중치가 부여된다. 이러한 가중치는 각 뉴런의 활성와 함수에 추가된 고유한 편향과 함께 머신 런닝을 통해 최적화해야 하는 매개변수이다.

역전파 알고리즘을 사용하면 각 개별 노드가 손실 함수의 전체 출력에 어떻게 기여하는지 계산할 수 있으므로 경사하강법 알고리즘을 통해 수백, 수억 개의 모델 가중치를 개별적으로 최적화할 수 있다. 최적의 결과를 얻는데 필요한 업데이트의 양과 세분성으로 인해 딥러닝에는 기존 머신러닝에 비해 매우 많은 양의 데이터와 컴퓨팅 리소스가 필요하다.

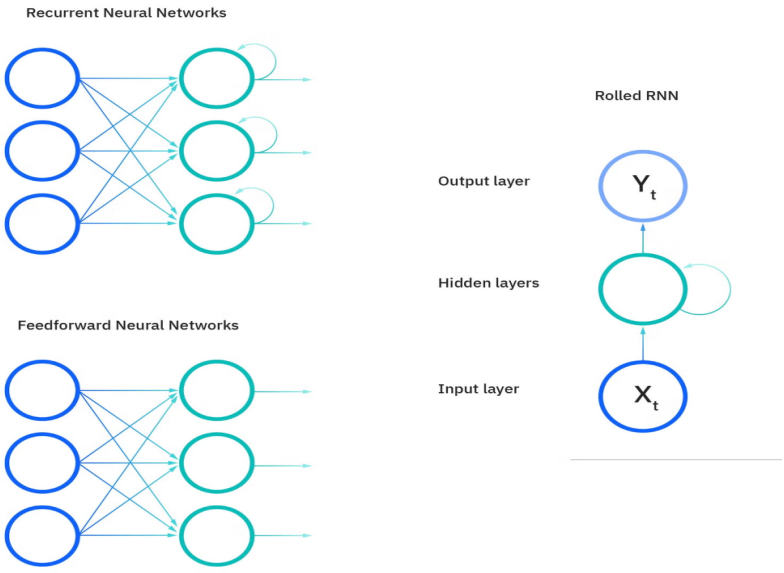
* CNN

CNN 컨볼루션 신경망은 시경망에 컨볼루션(합성곱: 두 함수를 섞어 새로운 함수나 신호를 만듦) 계층을 추가한다. CNN에서 컨볼루션 레이어는 가중치 필터를 적용하여 데이터에서 중요한 특징을 추출하는데 사용된다. CNN은 주로 컴퓨팅 비전 모델 및 이미지 데이터와 관련이 있다.



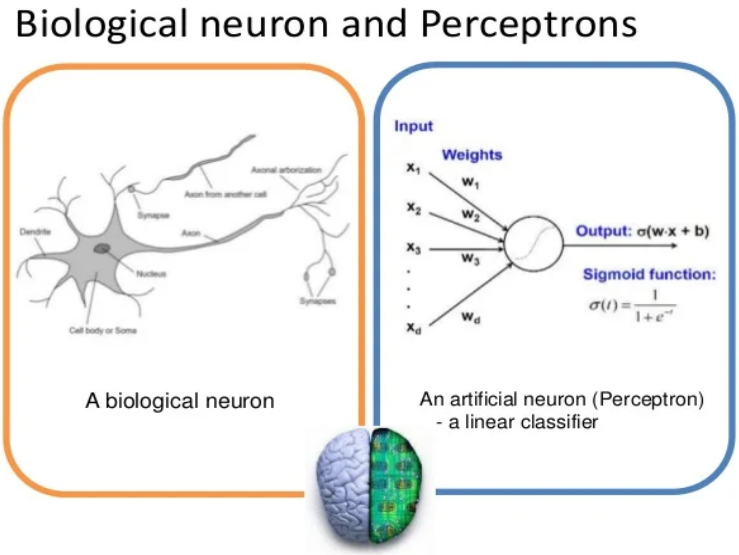
* RNN

RNN 순환신경망은 순차적 데이터를 처리하도록 설계되었다. 기존의 피드포워드 신경망은 단일 입력을 단일 아웃풋에 매핑하는 반면 RNN은 입력 시퀀스의 특정 단계에 대한 아웃풋이 다음 단계의 계산에 대한 입력으로 사용되는 순환루프에서 작동하여 입력 시퀀스를 아웃풋에 매핑한다.



4. 퍼셉트론

인공 신경망 모형의 하나인 퍼셉트론은 1957년에 Rosenblatt라는 사람에 의해서 처음 고안되었다. 일반적으로 인공 신경망 시스템은 동물의 신경계를 본따 만들었기 때문에 개념적으로나 그 형태로나 비슷한 부분이 많다.



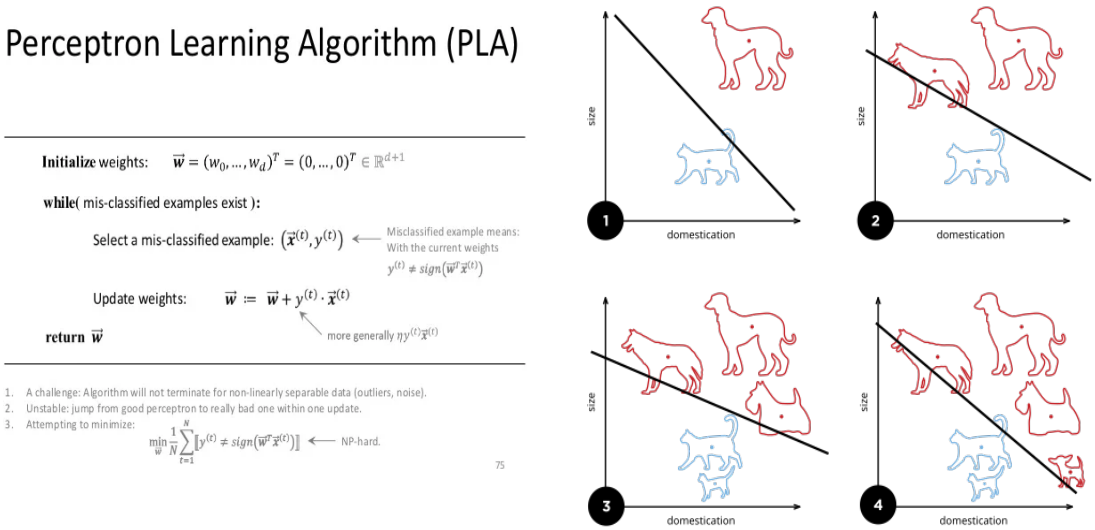
퍼셉트론은 다수의 신호(Input)을 입력받아서 하나의 신호(Output)을 출력한다. 이는 뉴런이 전기신호를 내보내 정보를 전달하는 것과 비슷하다. 그리고 뉴런의 수상돌기나 축색돌기처럼 신호를 전달하는 역할을 퍼셉트론에서는 weight가 그 역할을 한다. 가중치라고 부르는 이 weight는 각각의 입력신호에 부여되어 입력신호와의 계산을 하고 신호의 총합이 정해진 임계값(세타)을 넘었을 때 1을 출력한다. (이를 뉴련의 활성화activation 으로도 표현) 넘지 못하면 0 또는 -1을 출력한다.
각 입력신호에는 고유한 weight가 부여되며 weight가 클수록 해당 신호가 중요하다고 볼 수 있다. 여기서 기계학습이 하는 일은 이 weight(입력을 조절하니 매개변수로도 볼 수 있음)의 값을 정하는 작업이라고 할 수 있다.학습 알고리즘에 따라 방식이 다를 뿐 이 weight를 만들어내는 것이 학습이라는 차원에서는 모두 같다고 할 수 있다.

Output

$$w_1x_1 + w_2x_2 + \ldots + w_nx_n > \theta \Rightarrow 1$$

$$w_1x_1 + w_2x_2 + \ldots + w_nx_n \leq \theta \Rightarrow 0$$

퍼셉트론의 출력 값은 앞에서 말했듯이 1 또는 0(or -1)이기 때문에 선형 분류(linear classifier) 모형이라고도 볼 수 있다. 보통 실수형의 입력 벡터를 받아 이들의 선형조합을 계산하는 것이다.
처음에는 임의로 설정된 weight로 시작한다.
학습 데이터를 퍼셉트론 모형에 입력하며 분류가 잘못됐을 때 weight를 개선해 나간다.
weight를 개선해 나간다는 의미는 우리가 수학 문제를 잘못 풀었을 때 선생님이 다시 풀어오라고 하면 정답에 맞게 풀기 위해서 다시 풀고 다시 풀고 하다가 정답을 맞추는 것과 비슷하다. 그래서 학습이라고 부른다.



퍼셉트론은 모든 학습 데이터를 정확히 분류시킬 때까지 학습이 진행되기 때문에 학습 데이터가 선형적으로 분리될 수 있을 때 적합한 알고리즘이다. 이와 관련된 퍼셉트론 모형이 가지는 한계는 후에 설명하겠다. 선형분류는 위 그림과 같이 선으로 분류하는 것을 의미한다. 학습이 반복될수록 선의 기울기가 달라지는 것을 볼 수 있다. 학습을 하면서 weight가 계속 조정되는 것이다.
앞의 퍼셉트론 수식에서 나오는 세타θ를 -b로 치환하여 좌변으로 넘기면

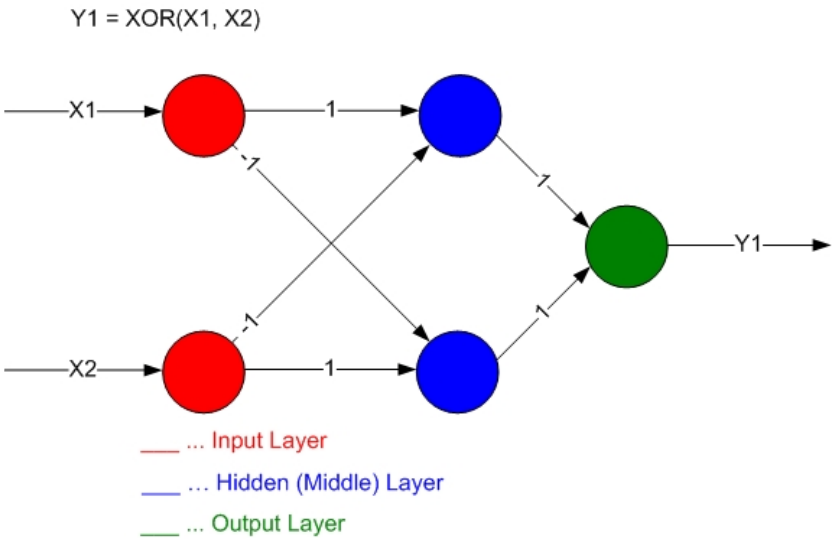
b + w1x1 + w2x2 <0 => 0
b + w1x1 + w2x2 >=0 => 1

과 같이 되며 여기에서 b를 편향(bias)라고 할 수 있다. 기계학습 분야에서는 모델이 학습 데이터에 과적합(overfitting)되는 것을 방지하는 것이 중요하다. 여기서 과적합이라 함은 모델이 엄청 유연해서 학습 데이터는 귀신같이 잘 분류하지만, 다른 데이터를 넣어봤을 때는 제대로 성능을 발휘하지 못하는 것을 말한다. 어느 데이터를 넣어도 일반적으로 잘 들어맞는 모델을 만드는 것이 중요하다.

따라서 앞에서 설명했듯이 편향은 세타로 학습 데이터(Input)이 가중치와 계산되어 넘어야 하는 임계점으로 이 값이 높으면 높을 수록 그만큼 분류의 기준이 엄격하다는 것을 의미한다. 그래서 편향이 높을 수록 모델이 간단해지는 경향이 있으며 (변수가 적고 더 일반화 되는 경우) 오히려 과소적합(underfitting)의 위험이 발생하게 된다. 반대로 편향이 낮을수록 한계점이 낮아 데이터의 허용범위가 넓어지는 만큼 학습 데이터에만 잘 들어맞는 모델이 만들어질 수 있으며 모델이 더욱 복잡해질 것이다. 허용범위가 넓어지는 만큼 필요 없는 노이즈가 포함될 가능성도 높다. 이를 편향과 분산의 트레이드오프 관계라고 보통 부른다.

퍼셉트론의 한계는 선형으로 분류를 할 수 있지만 XOR와 같이 선형 분류만 가능하며 비선형 분류는 불가능하다는 점이다. 직선 하나로 나눈 영역만 표현할 수 있어 XOR과 같은 데이터 형태는 분류가 불가능하다는 한계가 있다.

단일 퍼셉트론으로는 XOR을 분류할 수 없지만, 다층 퍼셉트론을 만들면 이를 극복할 수 있습니다. 다층(multi-layer)이라는 말은 하나의 퍼셉트론에 또 다른 퍼셉트론을 덧붙인다는 의미로 볼 수 있다. 단층 퍼셉트론이 비선형 영역을 분리할 수 없다는 것이 문제이며 다층으로 할 경우 비선형으로 이를 해결할 수 있다.

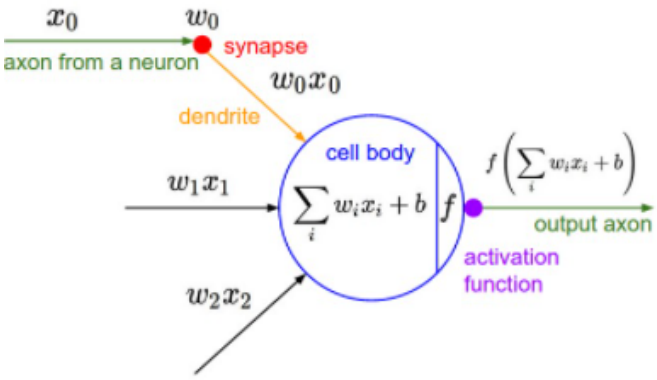


이런식으로 층을 겹겹이 쌓아나가면서 선형 분류만으로는 풀지 못했던 문제를 비선형적으로 풀 수 있게 된다.

PyTorch에는 퍼셉트론 클래스가 따로 없어서 가중합 z = Wx + b를 계산하는 부분이 nn.Linear이고, 여기에 계단 함수를 붙이면 퍼셉트론이 된다.

4. 활성화 함수

입력 신호의 총합을 출력 신호로 변환하는 함수를 일반적으로 활성화 함수라고 한다.
입력 신호의 총합이 활성화를 일으키는지를 정하는 역할.
아래는 신경망 그림으로 가중치(w)가 달린 입력 신호(x)와 편향(b)의 총합을 계산하고 함수 f에 넣어 출력하는 흐름을 보여준다.



신경망의 활성화 함수는 비선형 함수를 사용해야 한다. 선형 함수란 출력이 입력의 상수 배만큼 변화는 함수(1개의 곧은 직선)이고 비선형 함수는 선형이 아닌 함수로 직선 하나로는 그릴 수 없는 함수이다. 신경망에서 선형 함수를 이용하면 신경망의 층을 깊게하는 의미가 없어진다. 선형 함수의 문제는 층을 아무리 깊게해도 은닉층이 없는 네트워크로도 똑같은 기능을 할 수 있다.

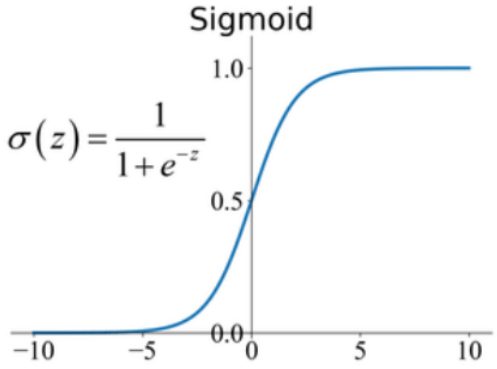
만약 h(x) = cx 를 활성화 함수로 사용한 3층 신경망이 있을 때,

이를 식으로 나타내면 y(x)= h(h(h(x))) 이다.

이 계산은 y(x)=c*c*c*x 를 수행하지만 사실 y(x)=ax 와 같다.

즉, 은닉층이 없는 네트워크로 표현 가능한 것이다.

이처럼 선형 함수인 활성화 함수를 이용하면 여러층으로 구성하는 이점을 살릴 수 없기 때문에 층을 쌓기 위해서는 비선형 함수인 활성화 함수를 사용해야 하고, 활성화 함수로는 비선형 함수를 사용해야 한다.



시그모이드(sigmoid)란 'S자 모양'이라는 뜻

실수 값을 입력받아 0~1 사이의 값으로 압축한다.

큰 음수 값일 수록 0에 가까워지고 큰 양수 값일 수록 1이 된다.

단점이 몇가지 있다.

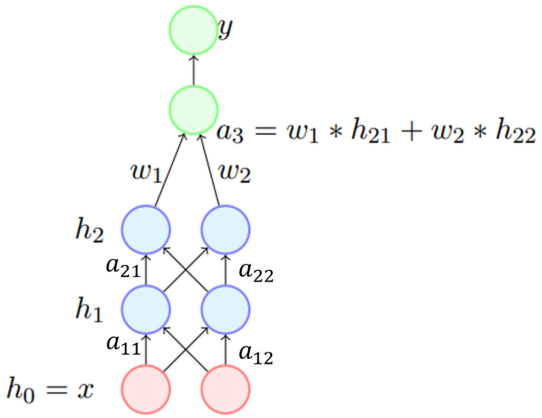
기울기 소멸 문제 (Vanishing Gradient Problem)

시그모이드 함수의 기울기는 입력이 0일 때 가장 크고, |x| 가 클수록 기울기는 0에 수렴한다. 이는 역전파 중에 이전의 기울기와 현재 기울기를 곱하면서 점점 기울기가 사라지게 된다. 그렇게 되면 신경망의 학습 능력이 제한되는 포화(Saturation)가 발생한다.

시그모이드 함숫값은 0이 중심 (zero-centered)이 아니다.

학습 속도가 느려진다.

만약 x가 입력이고 활성화 함수는 sigmoid를 사용하는 신경망을 그림으로 나타내면 아래와 같다.



각 노드의 아웃풋

a1 = w1*h01 + w2*h1

a2 = w1*h11 + w12*h1

a3 = w1*h21 + w22*h1

활성화 함수 적용 후의 output은 h1, h2
이때 h21, h22의 값은 시그모이드를 통과한 상태이기 때문에 0~1 사이로 둘 다 양수. 그렇기 때문에 빨간색 글씨의 공통 부분이 양수인 경우에는 기울기 값이 무조건 양수가 되고 빨간색 글씨의 공통 부분이 음수인 경우에는 기울기 값이 무조건 음수가 된다.

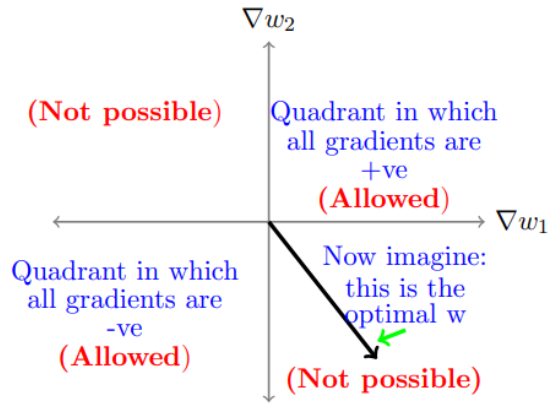
- Consider the gradient w.r.t. w_1 and w_2

$$\nabla w_1 = \frac{\partial \mathscr{L}(\mathbf{w})}{\partial y}\frac{\partial y}{h_3}\frac{\partial h_3}{\partial a_3}\ h_{21}$$

$$\nabla w_2 = \frac{\partial \mathscr{L}(\mathbf{w})}{\partial y}\frac{\partial y}{h_3}\frac{\partial h_3}{\partial a_3}\ h_{22}$$

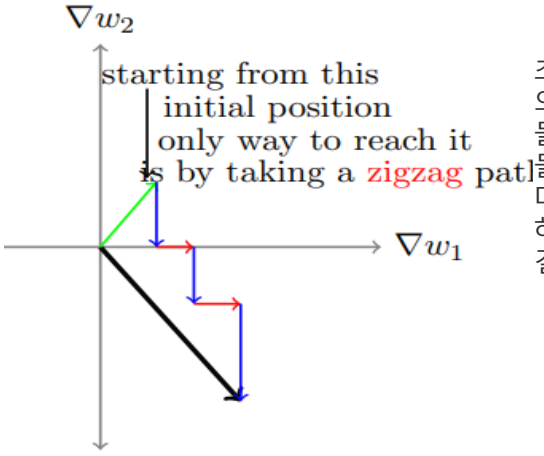
이 신경망의 가중치 기울기 계산

- Note that h_{21} and h_{22} are between $[0, 1]$ (*i.e.*, they are both positive)
- So if the first common term (in red) is positive (negative) then both ∇w_1 and ∇w_2 are positive (negative)
- This restricts the possible update directions



신경망이 시그모이드를 활성화 함수로 사용했을때 출력으로 가능한 기울기의 부호를 평면 위에 나타낸 그림

초록 화살표가 최적의 가중치이고 검은 화살표가 최적화를 위한 빠른 길을 의미



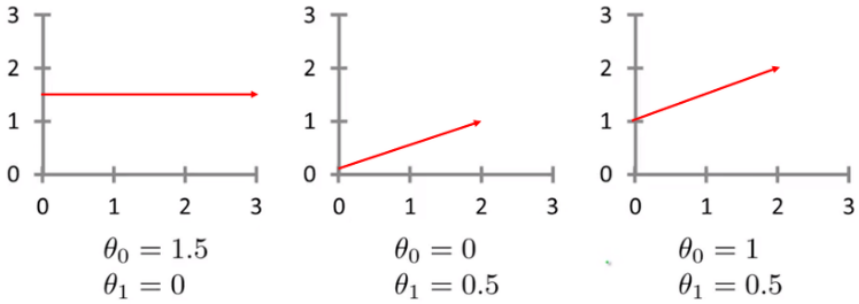
초기화된 가중치에서 업데이트를 시작한다면 w1의 기울기와 w2의 기울기가
둘 다 양수인 경우일직선으로 오른쪽으로 갔다가 둘 다 음수인 경우에는 일직선으로 내려가게 된다. 이를 반복하면서 대각선으로 곧장 향하지 못하고 지그재그로 수렴하게 되어 최적화가 오래 걸리는 문제가 생긴다.

PyTorch에서 활성화 함수는 두 가지 방식으로 사용할 수 있다. 첫 번째는 torch.nn 모듈의 클래스로, 모델 안에 하나의 층처럼 넣어서 사용한다. 두 번째는 torch나 torch.nn.functional(F)의 함수로, 값에 바로 적용할 때 사용한다.
Sigmoid는 nn.Sigmoid 클래스와 torch.sigmoid 함수가 있다. Tanh는 nn.Tanh 클래스와 torch.tanh 함수가 있다. ReLU는 nn.ReLU 클래스와 torch.relu, F.relu 함수가 있다. Leaky ReLU는 nn.LeakyReLU 클래스와 F.leaky_relu 함수가 있으며, 음수 구간의 기울기를 negative_slope 인자로 정한다. 기본값은 0.01이다.
분류 문제의 출력층에서는 nn.Softmax와 torch.softmax를 사용해 출력값을 확률로 바꾼다.
계단 함수는 미분값이 거의 모든 곳에서 0이라 역전파로 학습할 수 없기 때문에 nn 모듈에 층으로 존재하지 않는다. 함수로는 torch.heaviside만 제공된다.

6. 비용함수

비용함수는 원래의 값과 가장 오차가 작은 가설함수 를 도출하기 위해 사용되는 함수

$$h_\theta(x) = \theta_0 + \theta_1 x$$



가설함수의 형태를 결정짓는 것은 매개변수(parameter) 라 부르는 θ 이다.
θ값을 적절하게 조정하여 실제값 y에 가장 근접한 가설함수 를 Training set을 이용하여 도출해야 한다.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)} - y^{(i)})^2$$

이것이 Cost Function 비용함수이다,
h(x)-θ 은 가설함수와 실제 y값의 차이이다. 이것을 최소화 하는 것이 목표이지만 그냥 사용하면 오차가 양수 혹은 음수가 될 수 있으므로 제곱을 해준다. 그리고 Training set은 1부터 m까지 존재하기에 각각의 차이를 모두 더하여 평균을 내어 이 평균이 최소가 되게 만드는 θ를 구하는 것이 Cost Function의 목적이다. 이때 m이 아닌 2m으로 나눈 이유는 미분을 했을 때 내려오는 2와 자연스럽게 나눠지게 하기 위함이다.

$$h_\theta(x) = \theta_1 x \qquad J(\theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (\theta_1 x^{(i)} - y^{(i)})^2$$

- $If)\ \theta_1 = 1 \quad h_\theta(x) = x \quad J(\theta_1) = 0$
- $If)\ \theta_1 = 0.5 \quad h_\theta(x) = 0.5x \quad J(\theta_1) \approx 0.58$
- $If)\ \theta_1 = 0 \quad h_\theta(x) = 0 \quad J(\theta_1) \approx 2.3$

위는 (1,1), (2,2), (3,3) 3개의 Training set을 갖을 때 θ가 변함에 따라 어떻게 비용이 변하는지를 알려주고 있다.

MSE는 정답과 예측값의 차이를 제곱해서 평균을 낸 것이다. 예를 들어 정답이 [3, −0.5, 2]이고 예측값이 [2.5, 0, 2]라면 각 오차의 제곱은 0.25, 0.25, 0이 되고 이를 평균내면 약 0.1667이 된다. 오차를 제곱하기 때문에 오차가 클수록 벌을 훨씬 크게 준다. 또 곡선 형태라 모든 지점에서 미분이 매끄럽게 되어 경사하강법에 사용하기 좋아 가장 많이 쓰인다. 대신 제곱 때문에 다른 데이터와 동떨어진 이상치가 하나만 있어도 전체 비용이 크게 흔들리는 단점이 있다.
MAE는 오차의 절댓값을 평균낸 것이다. 같은 예시로 계산하면 (0.5 + 0.5 + 0) / 3 = 약 0.3333이 된다. 오차를 제곱하지 않고 그대로 더하기 때문에 이상치에 덜 민감하다. 하지만 절댓값 함수는 0에서 뾰족하게 꺾여 있어 미분이 되지 않는다는 단점이 있다.
분류 문제에서는 주로 교차 엔트로피를 사용한다.

정답이 0 또는 1 두 가지인 이진 분류에서는 Binary Cross Entropy를 사용한다.

$$BCE = -[\, y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \,]$$

예를 들어 정답이 1일 때 모델이 0.9라고 예측하면 손실은 −log(0.9) = 약 0.105로 작다. 반대로 0.1이라고 예측하면 −log(0.1) = 약 2.303으로 크게 증가한다. 즉 정답에서 멀어질수록 손실이 급격하게 커져서 확실하게 틀린 예측에 강한 벌을 준다.

클래스가 3개 이상인 다중 분류에서는 Categorical Cross Entropy를 사용한다.

$$CE = -\Sigma\, yk \log(\hat{y}k)$$

출력층에서 softmax로 각 클래스의 확률을 구한 뒤 정답 클래스의 확률에만 −log를 취하는 것과 같다. 정답 클래스에 높은 확률을 줄수록 손실이 0에 가까워진다.

분류 문제에서 MSE 대신 교차 엔트로피를 쓰는 이유도 있다. 앞의 활성화 함수에서 설명한 것처럼 시그모이드는 출력이 0이나 1 근처일 때 기울기가 0에 가까워진다. 이때 MSE를 쓰면 크게 틀린 상황에서도 기울기가 작아 학습이 느려진다. 교차 엔트로피는 log가 시그모이드의 포화를 상쇄해주기 때문에 틀릴수록 기울기가 커져 학습이 빠르게 진행된다.

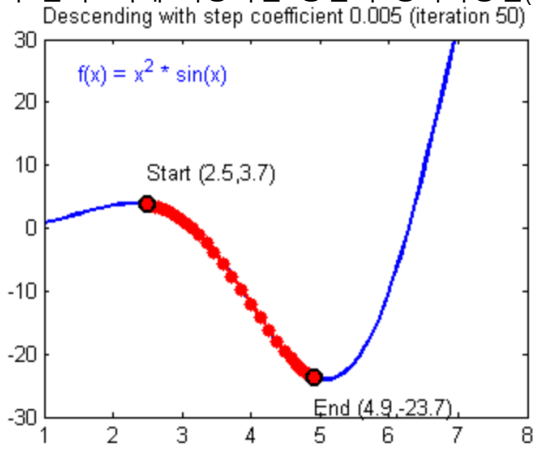
YOLO 같은 객체 검출 모델은 하나의 비용 함수만 쓰지 않고 여러 손실을 합쳐서 사용한다. 박스의 위치와 크기가 얼마나 맞는지를 나타내는 박스 손실, 그 위치에 물체가 있는지를 나타내는 객체성 손실, 어떤 물체인지를 나타내는 클래스 손실을 더해서 전체 비용을 만든다.

PyTorch의 비용 함수는 torch.nn 모듈에 클래스로 정의되어 있다. 먼저 객체를 만든 뒤 (예측값, 정답값)을 넣어 호출하면 손실 값이 나온다. 회귀 문제에서 MSE는 nn.MSELoss, MAE는 nn.L1Loss를 사용한다. 이진 분류에서는 nn.BCELoss와 nn.BCEWithLogitsLoss를 사용한다. nn.BCELoss는 sigmoid를 거친 확률값을 입력으로 받고, nn.BCEWithLogitsLoss는 sigmoid 이전의 값을 받아 내부에서 sigmoid까지 함께 계산하기 때문에 수치적으로 더 안정적이다. 다중 분류에서는 nn.CrossEntropyLoss를 사용한다. 이 클래스는 내부에 softmax가 포함되어 있어서 모델 출력에 softmax를 따로 적용하지 않은 값을 넣어야 하고, 정답은 클래스 번호로 넣는다.

비용 함수 클래스에는 reduction 인자가 있다. reduction='none'이면 데이터 하나하나의 손실을 그대로 반환하고, 'mean'(기본값)이면 평균을, 'sum'이면 합을 반환한다. 앞에서 설명한 손실 함수와 비용 함수의 구분이 이 인자로 구현되어 있다고 볼 수 있다.

7. 경사하강법

비용 함수로 모델이 얼마나 틀렸는지를 알았다면 이제 비용을 줄이는 방향으로 가중치를 바꿔야 한다. 이때 사용하는 방법이 경사하강법(Gradient Descent)이다.



비용 함수를 가중치에 대한 그래프로 그리면 산처럼 높은 곳과 낮은 골짜기가 있는 모양이 된다. 경사하강법은 산 위에서 눈을 가리고 가장 낮은 골짜기를 찾아 내려가는 것에 비유할 수 있다. 앞이 보이지 않으니 발밑의 기울기를 느끼고 내려가는 방향으로 한 걸음씩 이동하는 것이다. 여기서 발밑의 기울기가 비용 함수의 미분값이다.

$$w \leftarrow w - \eta \cdot \partial L/\partial w$$

기울기가 양수라면 w가 커질수록 비용이 커진다는 뜻이므로 w를 줄인다. 기울기가 음수라면 반대로 w를 키운다. 즉 항상 기울기의 반대 방향으로 이동한다. 여기서 에타는 학습률로, 한 번에 얼마나 크게 이동할지를 정하는 보폭이다.

경사하강법은 기울기를 계산할 때 데이터를 얼마나 사용하느냐에 따라 세 가지로 나뉜다.
배치 경사하강법(Batch GD)은 전체 데이터를 모두 사용해 기울기를 한 번 계산하고 가중치를 한 번 갱신한다. 방향은 정확하지만 데이터가 많으면 한 번 갱신하는 데 시간이 오래 걸리고 메모리도 많이 필요하다.

확률적 경사하강법(SGD, Stochastic GD)은 데이터 하나마다 기울기를 계산하고 가중치를 갱신한다. 갱신은 빠르지만 데이터 하나만 보기 때문에 방향이 들쭉날쭉하다.

미니배치 경사하강법은 데이터를 32개, 64개 같은 작은 묶음(배치)으로 나눠 묶음마다 갱신한다. 두 방식의 장점을 적절히 섞은 방식으로 실제로 가장 많이 사용된다.

여기서 전체 데이터를 한 바퀴 다 학습하는 것을 1 에폭(epoch)이라고 하고, 가중치를 한 번 갱신하는 것을 1 이터레이션(iteration)이라고 한다. 예를 들어 데이터가 6400개이고 배치 크기가 64라면 1 에폭은 100 이터레이션이 된다.

기본 경사하강법에는 한계도 있다. 비용 함수가 여러 개의 골짜기를 가지고 있으면 가장 낮은 곳이 아닌 근처의 작은 골짜기(지역 최솟값)에 빠질 수 있다. 또 기울기가 0이지만 최솟

값이 아닌 안장점에서 멈출 수도 있다. 앞에서 본 시그모이드의 지그재그 문제처럼 방향이 계속 흔들려 수렴이 느려지는 경우도 있다.

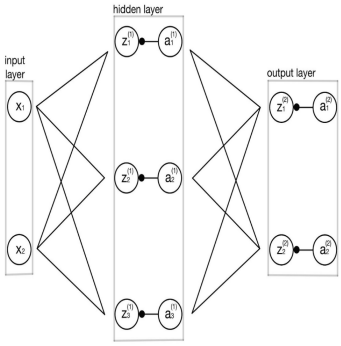
이런 문제를 개선하기 위해 여러 옵티마이저가 만들어졌다. Momentum은 공이 굴러가듯 이전에 이동하던 방향을 관성처럼 기억해서 반영한다. 그래서 흔들림이 줄고 작은 골짜기를 넘어갈 수 있다. Adam은 Momentum의 관성 개념에 더해 가중치마다 학습률을 자동으로 조절하는 방식을 합친 것으로, 현재 가장 많이 사용되는 옵티마이저이다.

PyTorch에서 경사하강법은 torch.optim 모듈의 옵티마이저 클래스로 사용한다. 기본 경사하강법은 torch.optim.SGD를 사용하며, model.parameters()로 학습할 가중치들을 넘기고 lr로 학습률을 지정한다. momentum 인자를 주면 SGD+Momentum이 되고,
Adam은 torch.optim.Adam을 사용한다. 옵티마이저에서 주로 쓰는 함수는 두 가지이다. optimizer.zero_grad()는 이전에 계산된 기울기를 0으로 초기화하고, optimizer.step()은 계산된 기울기를 이용해 가중치를 갱신한다.
배치 GD, 미니배치 SGD, SGD의 구분은 옵티마이저가 아니라 torch.utils.data의 DataLoader 클래스에서 정해진다. batch_size를 전체 데이터 수로 하면 배치 GD, 64 같은 값으로 하면 미니배치 SGD, 1로 하면 SGD가 된다.

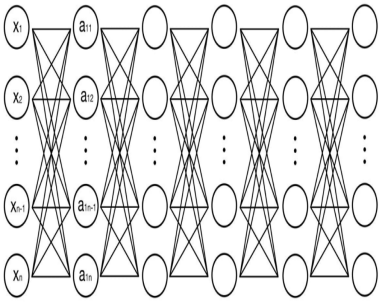
8. 역전파

경사하강법으로 가중치를 갱신하려면 모든 가중치에 대해 기울기를 알아야 한다. 하지만 신경망에는 가중치가 매우 많다. 뒤에서 볼 784−128−10 신경망만 해도 학습해야 할 숫자가 101,770개이다. 이 기울기를 하나씩 따로 계산하면 너무 오래 걸린다. 이를 효율적으로 계산하는 방법이 역전파(Backpropagation)이다.

역전파는 출력층에서 계산된 오차를 뒤에서 앞으로 거꾸로 전달하면서 각 가중치의 기울기를 구하는 방법이다. 핵심 원리는 미분의 연쇄 법칙(Chain Rule) 하나이다. 연쇄 법칙은 여러 함수가 합성되어 있을 때 전체의 미분은 각 함수의 미분을 곱한 것과 같다는 법칙이다.



역전파



순전파

뉴런 하나에서 입력 x가 들어와 z = wx + b를 계산하고, 활성화 함수를 거쳐 a = f(z)가 되고, 이 a로 손실 L을 계산한다고 해보자. w가 L에 미치는 영향을 한 번에 구하기는 어렵다. 하지만 w가 z에, z가 a에, a가 L에 미치는 영향을 각각 구해서 곱하면 쉽게 구할 수 있다.

$$\frac{\partial L_1}{\partial w_{i1}^{(1)}} = \frac{\partial L}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{i1}^{(1)}}$$

역전파를 계산 그래프로 보면 각 노드는 자기 일만 하면 된다. 노드는 뒤에서 들어온 기울기(upstream gradient)에 자기의 국소 미분(local gradient)만 곱해서 앞으로 넘긴다. 전체 식을 알 필요가 없는 것이다. 예를 들어 덧셈 노드는 들어온 기울기를 그대로 양쪽으로 전달하고, 곱셈 노드는 들어온 기울기에 상대편 입력값을 곱해서 전달한다.

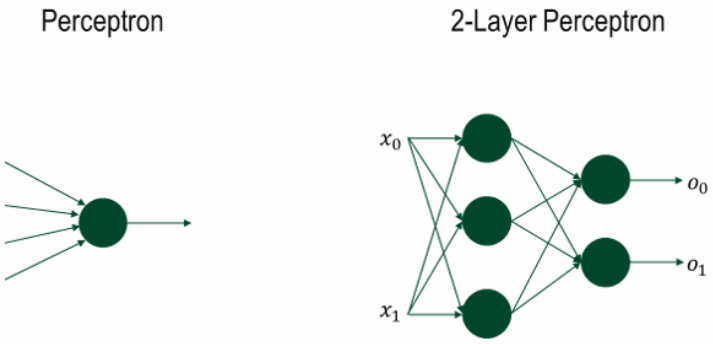
역전파를 "역"방향으로 하는 이유는 뒤쪽 층에서 이미 구한 기울기를 앞쪽 층이 그대로 재사용할 수 있기 때문이다. 층이 여러 개일 때 앞쪽 가중치의 기울기를 구하려면 뒤쪽의 미분값들이 모두 필요하다. 이미 뒤에서 계산해둔 값을 이어서 곱하기만 하면 되므로 중복 계산이 없다. 그래서 파라미터가 수백만 개여도 순전파 1번과 역전파 1번이면 모든 기울기를 구할 수 있다.

역전파는 앞에서 설명한 기울기 소멸 문제와도 직접 연결된다. 역전파는 층을 지날 때마다 미분값을 곱하는데, 시그모이드의 미분값은 최대 0.25이다. 층이 10개라면 최대로 잡아도 0.25를 10번 곱하게 되어 약 0.00000095가 된다. 결국 입력층에 가까운 가중치에는 기울기가 거의 전달되지 않아 학습이 되지 않는다. ReLU는 양수 구간에서 미분값이 1이라 곱해도 줄어들지 않기 때문에 깊은 신경망에서 기본으로 사용된다.

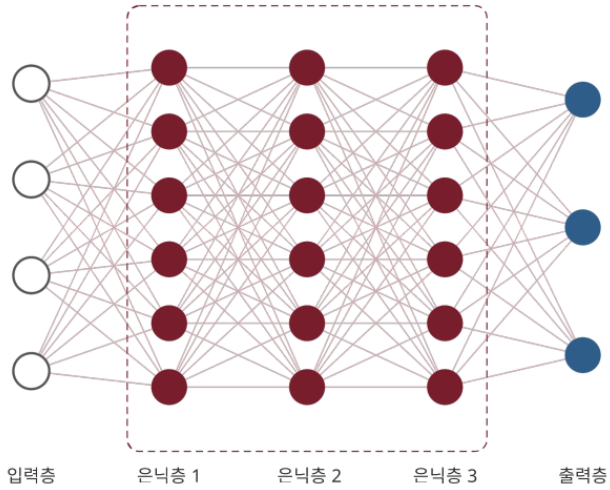
PyTorch는 autograd라는 자동 미분 기능으로 역전파를 수행한다. 텐서를 만들 때 requires_grad=True로 설정하면 그 텐서로 하는 모든 연산이 계산 그래프로 기록된다. nn.Linear 같은 층의 weight와 bias는 기본적으로 이 설정이 켜져 있다. 순전파로 손실 값을 구한 뒤 loss.backward() 함수를 호출하면, 기록된 계산 그래프를 따라 연쇄 법칙이 적용되어 각 가중치의 기울기가 계산된다. 이 값은 각 텐서의 .grad 속성에 저장되고, 옵티마이저가 이 값을 사용해 가중치를 갱신한다.

9. 심층신경망

심층 신경망은 Deep Neural Network에 해당하며 여러개의 은닉층을 가지고 있다. 이는 MLP(Multi-layer Perceptron)의 확장된 형태라고 보면된다. 모든 DNN은 MLP 구조를 기반으로 하지만, MLP 중 은닉층이 많은 구조를 DNN이라고 생각하면 된다.



앞에서 설명한 퍼셉트론(뉴런)을 여러 층으로 쌓은 인공신경망이다. 퍼셉트론 장에서 단층으로는 XOR을 풀 수 없지만 층을 쌓으면 풀 수 있다고 했는데, 이 층을 더 많이 쌓은 것이 심층신경망이다. 보통 은닉층이 2개 이상이면 "깊다(deep)"고 부르며, 이런 신경망을 학습시키는 것을 딥러닝이라고 한다.



심층신경망은 세 종류의 층으로 구성된다. 입력층은 데이터를 받는 층으로, 이미지라면 각 픽셀 값이 입력이 된다. 은닉층은 입력에서 특징을 뽑고 조합하는 층이다. 출력층은 최종 예측을 내는 층으로, 분류라면 각 클래스의 확률, 위치 예측이라면 좌표가 출력된다. 한 층의 모든 뉴런이 다음 층의 모든 뉴런과 연결된 층을 완전연결층(Fully Connected Layer, Dense Layer)이라고 한다.

층을 깊게 쌓으면 좋은 이유는 층마다 점점 더 복잡한 특징을 학습할 수 있기 때문이다. 이미지를 예로 들면 앞쪽 층은 선이나 모서리 같은 단순한 특징을 찾는다. 중간 층은 이를 조합해 원이나 모서리 모양 같은 형태를 찾고, 뒤쪽 층은 이를 다시 조합해 공이나 골대 같은 물체를 알아본다. 단, 앞에서 설명했듯이 층 사이에 비선형 활성화 함수가 있어야만 이런 효과가 생긴다.

입력층에서 은닉층을 거쳐 출력층 순서로 값을 계산해 나가는 과정을 순전파(Forward Propagation)라고 한다. 층 하나의 계산은 다음 두 줄로 표현된다.

$$z(l) = W(l)\, a(l-1) + b(l)$$
$$a(l) = f(z(l))$$

여기서 W(l)은 l번째 층의 가중치 행렬로, 크기는 (출력 뉴런 수 × 입력 뉴런 수)이다. a(0)는 입력 x이고, 마지막 층의 a가 최종 예측값이 된다. 즉 이전 층의 출력에 가중치를 곱하고 편향을 더한 뒤 활성화 함수를 통과시키는 과정을 층마다 반복하는 것이다.

심층신경망의 파라미터 개수를 직접 세어보자. 28×28 흑백 손글씨 이미지(MNIST)를 입력받아 0~9 숫자를 분류하는 784−128−10 신경망이 있다고 하자. 첫 번째 층(784 → 128)은 가중치 784 × 128 = 100,352개와 편향 128개로 100,480개이다. 두 번째 층(128 → 10)은 가중치 128 × 10 = 1,280개와 편향 10개로 1,290개이다. 합치면 총 101,770개의 숫자를 학습해야 한다. 은닉층 하나짜리 작은 신경망도 이 정도이기 때문에 앞에서 설명한 역전파가 반드시 필요하다.

심층신경망은 파라미터가 많아 표현력이 강한 만큼 앞에서 설명한 과적합이 일어나기 쉽다. 이를 막기 위해 학습 중에 일부 뉴런을 무작위로 꺼서 특정 뉴런에만 의존하지 않게 하는 드롭아웃(Dropout) 같은 방법을 사용한다.

PyTorch에서 신경망은 nn.Module 클래스를 기반으로 만든다. 가장 간단한 방법은 nn.Sequential 클래스로, 괄호 안에 층을 순서대로 나열하면 입력이 그 순서대로 통과한다. 예를 들어 nn.Flatten, nn.Linear(784, 128), nn.ReLU, nn.Linear(128, 10)을 순서대로 넣으면 앞에서 계산한 784−128−10 신경망이 된다. 이 모델의 파라미터 수를 세어보면 101,770개로 직접 계산한 값과 같다. nn.Flatten은 28×28 이미지를 784개짜리 한 줄로 펴주는 역할을 한다.